

VideoAsset Manualize

VideoAsset Manualize は、既存の研修動画・マニュアル動画を、再利用可能な教育資産・業務資産へ変換するプロジェクトです。

社内に蓄積された動画を、単なる視聴素材のまま放置するのではなく、文字起こし・手順抽出・要点整理・章立て・注意点整理を通じて、構造化JSON と PDFマニュアル に変換し、教育・引き継ぎ・標準化・検索・再利用に耐える状態へ整備します。

本プロジェクトは、既存の動画解析基盤の思想を継承しつつ、対象を「競合分析」から「研修・業務手順の資産化」へ拡張する取り組みです。

本READMEは、初期立ち上げ段階における 統合ドキュメント です。リポジトリ整備の進行に応じて、docs/ 配下の個別ドキュメントへ分割・再編する前提で設計しています。

1. プロジェクト概要

企業内には、研修動画、業務手順動画、操作説明動画、引き継ぎ用録画など、多くの動画資産が蓄積されています。

しかし、それらの多くは「見れば分かるはず」という前提で保管されており、実務上は検索しづらく、再利用しづらく、標準化にもつながりにくい状態にあります。

VideoAsset Manualize は、こうした既存動画を対象に、以下を一通貫で行うことを目的とします。

- 動画の取得
- 文字起こし
- 内容の構造化
- 手順抽出
- 注意点・NG例の整理
- チェックリスト化
- JSON化
- HTML/PDFマニュアル化

最終的には、動画1本ごとに「見るだけ」ではない、現場で使えるマニュアル資産として納品・再利用できる状態を作ります。

2. 背景と課題

多くの企業では、研修動画やマニュアル動画が社内ストレージ、共有ドライブ、ポータル、動画配信基盤に蓄積されています。一方で、以下のような課題が発生しています。

主な課題

- 動画はあるが、必要な情報をすぐ探せない
 - 目的の場面を探すために最初から見返す必要がある
 - 特定の操作や注意点をピンポイントで参照しづらい

- 教育運用にばらつきが出る
 - 「とりあえずこの動画を見ておいて」で教育が完結してしまう
 - 現場トレーナーごとに補足説明の質や粒度が異なる
- テキスト資産として整理されていない
 - 動画内にノウハウがあるが、手順書やFAQに転用しづらい
 - 他資料への転記や更新が手作業になりがち
- 文字起こし後の整理が進まない
 - 文字起こしはあるが、構造化・編集・章立て・要点整理が重い
 - 結果としてRAWテキストが放置される

課題の本質

問題は「動画がないこと」ではなく、**動画が業務資産として整備されていないこと**です。

そのため必要なのは、新規動画制作そのものよりも、既存動画を整理・構造化し、再利用可能な教育資産へ変換する仕組みです。

3. 目的

本プロジェクトの目的は、動画視聴前提の運用から脱却し、**テキスト + PDFマニュアルも選べる運用**へ移行することです。

目的一覧

- 既存の研修動画・マニュアル動画を、組織で使い回せる教育資産に変換する
 - 動画視聴だけに依存しない、参照性の高い運用へ移行する
 - 教育内容や業務手順の標準化を進める
 - 属人化した引き継ぎや口頭説明を減らす
 - JSONベースで、FAQ・検索・QAボット・ナレッジ基盤への二次利用を可能にする
-

4. 提供価値

VideoAsset Manualize が提供する価値は、単なる文字起こしや要約に留まりません。「動画を、使える教育・業務資産に変えること」そのものが価値です。

4.1 使える教材化

- 動画を「見る教材」から「参照できる教材」へ変換
- 必要な箇所だけを確認できる
- 章立て・手順化・注意点整理により実務利用しやすくなる

4.2 教育コスト削減

- 新人が毎回動画の最初から見直す必要を減らせる
- OJTで何度も同じ説明を繰り返す負担を軽減できる
- 「手順書 + 必要時のみ動画参照」という効率的な教育運用が可能になる

4.3 品質標準化

- 店舗、コールセンター、物流、製造、管理部門などで手順のばらつきを抑えられる
- テキストベースで横展開しやすくなる
- 同一業務に対する説明品質を均質化できる

4.4 ナレッジ資産化

- 動画内に埋もれていたノウハウを検索・編集・再利用可能な形に変換
- FAQ、チェックリスト、社内検索、QA支援の基礎データとして活用しやすくなる
- 将来的なナレッジベース構築の前処理として機能する

5. 本プロジェクトの立ち位置

本プロジェクトは、以下のいずれとも異なります。

これは何ではないか

- 動画制作会社の仕事ではない
 - 新しい動画を作ることが主目的ではない
- 単なる文字起こしサービスではない
 - テキスト化して終わりではない
- eラーニングシステム販売でもない
 - 箱を売るのではなく、箱に入れる中身を作る

これは何か

既にある動画を、現場で使える手順書レベルまで構造化する仕事です。

- 動画を解析し
- 実務上重要な情報を抽出し
- 再利用しやすいデータ構造に整理し
- 現場で読めるPDFやマニュアル素材に変換する

つまり、**動画資産の再編集・再設計・再利用基盤化**が、このプロジェクトの本質です。

6. 既存基盤の継承方針

本プロジェクトは、既存基盤 **video-insight-spec** の思想を継承します。継承する中心思想は、以下の一文に集約されます。

生データ → 構造化JSON → 複数形式へ展開

この思想により、動画から得られた情報を一度中間表現として正規化し、その後にHTML、PDF、要約、チェックリスト、FAQ候補など複数の成果物へ展開できます。

継承する考え方

- 動画を直接レポート化しない
- まず中間JSONを正本として確立する
- 表示形式とデータ構造を分離する
- 複数の出力フォーマットに再利用可能な基盤を作る
- AI出力も証跡とレビュー前提で扱う

流用・置換の整理

既存基盤の要素	VideoAsset Manualize での扱い
InsightSpecRepository	TrainingAssetRepository として転用
insight_spec_{id}.json	training_asset_spec_{id}.json に置換
competitive / growth / theme ビュー	manual / training / checklist / qa 系ビューに置換
ReportGenerator	マニュアル生成オーケストレータとして転用
HTMLFormatter	マニュアルHTMLテンプレート生成に転用
TextFormatter	RAWログ・要約・章別テキスト整形に転用
ExecutiveSummaryFormatter	ダイジェスト版・1枚サマリー生成に応用
NarrativeEngine	補足説明・注意点補強・初心者向け解説生成に転用
証跡DB / sidecar 的発想	発話区間・OCR・画面キャプチャ候補の保持に継承

7. アーキテクチャ概要

本プロジェクトでは、`training_asset_spec_{id}.json` のような最終統合JSONを中心に据えます。

このJSONを **Single Source of Truth** とし、すべての派生物はここから生成します。

全体像

1. 動画を取得
2. 音声認識・OCR・画面情報抽出を実施
3. 発話・画面証跡を統合
4. 説明パートと手順パートを分類
5. 手順を章・ステップに構造化
6. 注意点・NG例・チェック項目を抽出
7. `training_asset_spec_{id}.json` を生成
8. JSONからHTML/PDF/要約/チェックリスト等を出力

基本アーキテクチャ

動画ファイル / 動画URL
↓
文字起こし・OCR・画面解析
↓
証跡統合データ（発話・画面・時刻）
↓
手順抽出 / 要約分離 / 構造化
↓
`training_asset_spec_{id}.json`

8. データモデル方針

最終統合JSONは、用途ごとの責務分離を明確にしたレイヤー構造で設計します。

推奨JSONレイヤー構造

レイヤー	役割	主な内容
asset_meta	資産の基本情報	タイトル、対象部署、対象者、版、更新日、言語、ソース識別子
source_evidence	元動画に紐づく証跡	transcript、OCR、時刻、画面情報、話者、信頼度
instructional_core	実務で使う中心情報	目的、前提条件、章、手順、ステップ、条件分岐、注意点、NG例
derived_views	利用シーン別の見せ方	manual、training、checklist、qa、digest など
delivery_assets	納品物への変換情報	RAWログ、要約、PDF用データ、HTML用データ、冊子統合情報
_metadata	生成メタデータ	生成日時、モデル、バージョン、レビュー状態、更新履歴

レイヤー詳細

asset_meta

動画資産そのものの識別と管理を担当します。

想定項目:

- asset_id
- title
- source_video_id
- department
- target_audience
- language
- version
- updated_at
- owner
- tags

source_evidence

自動抽出結果の根拠を保持します。

後続のレビュー、品質確認、再生成、監査対応のために重要です。

想定項目:

- transcript_segments
- ocr_segments
- timeline_marks
- speaker_segments
- screen_capture_candidates
- confidence_scores

instructional_core

業務マニュアルの正本となる中核情報です。

想定項目:

- purpose
- target_users
- prerequisites
- chapters
- procedures
- steps
- warnings
- ng_examples
- common_mistakes
- checklist_items

derived_views

閲覧者や利用目的に応じて再構成したビューです。

例:

- manual_view
- training_view
- checklist_view
- qa_view
- supervisor_view

delivery_assets

実際に納品・配布する成果物の出力用データを保持します。

例:

- raw_transcript_text
- summary_text
- html_render_payload
- pdf_render_payload
- booklet_payload

_metadata

処理履歴・版管理・品質運用を支える情報を管理します。

例:

- generated_at
- pipeline_version
- model_info
- review_status
- approved_by
- change_log

最重要な仕様変更

本案件では、「要約」と「手順抽出」を同じ処理にしないことを最重要方針とします。

要約してよいもの

- 説明
- 背景
- メリット
- 意義
- 補足説明
- 雑談

潰してはいけないもの

- 作業手順
- 操作順
- ボタン名
- 入力項目名
- 条件分岐
- 注意点
- NG例
- よくあるミス
- チェック項目

つまり、

- 説明・背景は要約する
- 操作・条件分岐・ボタン名・注意点はステップとして保持する

この分離ができないと、現場で使えるマニュアルにはなりません。

推奨抽出階層

情報は以下の4階層で整理することを推奨します。

階層	役割	例
章	大きなまとまり	ログイン準備、申請作成、承認処理、エラー対応
手順	業務単位のまとまり	顧客登録、添付ファイル追加、申請送信
ステップ	実際の操作単位	「メニューを開く」「顧客管理を押す」「必須項目を入力」
補足	再現性と安全性を支える情報	注意点、NG例、分岐条件、よくあるミス、確認ポイント

9. 処理フロー

VideoAsset Manualize の処理フローは、動画を直接PDF化するのではなく、構造化を挟むパイプラインとして設計します。

基本フロー

1. 対象動画の受領

- ファイル受領またはURL指定
- 対象部署・用途・対象者を確認

2. 動画取得・前処理

- 動画ダウンロードまたはローカル読込
- 音声抽出
- フレーム抽出

3. 証跡生成

- 音声認識による文字起こし
- OCRによる画面上テキスト抽出
- 話者区分・時刻付与
- 画面キャプチャ候補抽出

4. 内容分類

- 説明・背景パート
- 実操作パート
- 注意喚起パート
- 例外処理パート
- 雑談・周辺説明

5. 構造化処理

- 章立て
- 手順抽出
- ステップ分割
- 条件分岐整理
- 注意点・NG例抽出
- チェックリスト生成

6. 統合JSON生成

- training_asset_spec_{id}.json を出力

7. 派生物生成

- RAW文字起こし
- 要約テキスト
- HTMLマニュアル
- PDFマニュアル
- チェックリスト
- FAQ候補

8. 人手レビュー

- 手順抜け確認
- 文言修正
- 注意点の追加
- 現場表現への調整

PDF生成方針

初期フェーズでは、**JSON → HTML → PDF** の流れを推奨します。

- ブラウザ上でのプレビュー確認がしやすい
 - HTMLテンプレートを再利用しやすい
 - レイアウト・デザイン調整が柔軟
 - 将来的なブランド対応や帳票強化がしやすい
 - PDF以外にWebプレビューとしても利用可能
-

10. 納品物

基本納品物

- **構造化JSON**
 - 動画ごとの統合データ
 - 将来的な再利用の正本
- **RAW文字起こしテキスト**
 - 元発話ログ
 - 監査・差分確認・再編集用
- **要約テキスト**
 - 概要説明用
 - 管理者・提案用サマリー
- **PDFマニュアル**
 - 目的
 - 対象者
 - 前提条件
 - 手順
 - 注意点
 - NG例
 - チェックリスト

オプション納品物

- 複数動画をまとめた**業務マニュアル冊子PDF**
- 各マニュアルをまとめた**ダイジェスト版ハンドブック**
- **企業ロゴ・ブランドカラー対応**
- チェックリスト単票
- FAQ候補一覧
- 新人向け簡易版
- 管理者向け確認版

証跡設計の重要性

本プロジェクトでは、生成物の信頼性を高めるため、以下の証跡情報を保持する方針を重視します。

証跡項目	保持する理由
start_ms / end_ms	ステップと元動画の対応箇所を追跡するため
OCR文字列	ボタン名・項目名・エラーメッセージの根拠確認のため
画面キャプチャ候補	PDFや手順書への図版挿入候補として使うため
話者ラベル	講師説明と実演者発話を区別するため
信頼度スコア	自動抽出結果のレビュー優先度を判定するため

証跡を持たない自動生成は、修正や確認のたびに動画を見直す必要があり、運用コストを増やします。

そのため本プロジェクトでは、**Evidence First** を設計原則として採用します。

11. MVP定義

初期段階では、範囲を絞ったMVPで価値検証を行います。

MVPの範囲

1動画 → 1構造化JSON → 1PDF → 1要約 → 1RAWログ

まずはこの単位で、技術・品質・運用の成立性を確認します。

MVPで評価する観点

評価項目	合格ライン
手順抜け	現場担当者が「大きな抜けはない」と判断できる
粒度	新人が読んで再現可能な粒度になっている
注意点	事故やミスにつながる注意点が落ちていない
根拠追跡	どの動画箇所から抽出したか確認できる

MVPでやらないこと

- 高度なデザインカスタマイズ
- 大規模ダッシュボード化
- 多言語展開
- 自動承認ワークフローの本格実装
- 大規模冊子統合

12. ロードマップ

Phase 0: 構想・要件整理

目的:

- プロジェクト定義
- 想定ユースケースの明確化
- 対象動画の種類整理
- 出力物イメージの合意形成

主な成果物:

- README / 概要資料
- 対象ユースケース一覧
- 初期スコープ定義

Phase 1: スキーマ定義・基盤設計

目的:

- training_asset_spec の初版定義
- 証跡設計の確立
- 抽出階層と分類ルールの固定
- テンプレート構成案の作成

主な成果物:

- JSONスキーマ初版
- 抽出ルール仕様
- 命名規則
- ディレクトリ構成
- HTML/PDFテンプレート骨格

Phase 2: MVP実装・パイロット検証

目的:

- 1動画単位での一連のパイプライン実装
- パイロット動画で品質検証
- 人手レビューを含む運用確認

主な成果物:

- 構造化JSON出力
- RAW文字起こし
- 要約テキスト
- PDFマニュアル
- レビュー反映済みの改善ログ

Phase 3: 商品化・拡張

目的:

- 複数動画対応
- 冊子化
- デザイン強化
- 運用テンプレート整備
- 提案・販売パッケージ化

主な成果物:

- 正式パック設計
- 複数動画統合PDF
- ダイジェスト版
- ブランド対応テンプレート
- 運用ガイド

実行ステップ初期案

1. 対象企業・対象部署の選定
 2. パイロット動画数本の受領
 3. 解析・構造化・PDF生成の一連フロー実行
 4. 現場担当者からのフィードバック取得
 5. 抽出プロンプト・テンプレート・出力フォーマットを調整
 6. 正式パックとして提供開始
-

13. 推奨ディレクトリ構成

```
video-asset-manualize/
├── README.md
├── docs/
│   ├── concept/
│   │   ├── PROJECT_CHARTER.md
│   │   ├── VALUE_PROPOSITION.md
│   │   └── USECASES.md
│   ├── architecture/
│   │   ├── SYSTEM_OVERVIEW.md
│   │   ├── PIPELINE_DESIGN.md
│   │   └── EVIDENCE_MODEL.md
│   ├── roadmap/
│   │   ├── MVP_PLAN.md
│   │   └── PHASES.md
│   └── templates/
│       ├── MANUAL_TEMPLATE_SPEC.md
│       └── PDF_LAYOUT_GUIDE.md
├── schemas/
│   ├── training_asset_spec.schema.json
│   ├── transcript_segment.schema.json
│   ├── procedure_step.schema.json
│   └── checklist_item.schema.json
├── pipeline/
│   ├── video_loader.py
│   ├── transcript_pipeline.py
│   ├── ocr_pipeline.py
│   ├── evidence_builder.py
│   ├── structure_extractor.py
│   └── training_asset_builder.py
├── converter/
│   ├── training_asset_repository.py
│   ├── html_formatter.py
│   ├── text_formatter.py
│   ├── pdf_generator.py
│   └── digest_formatter.py
├── generators/
│   ├── manual_generator.py
│   ├── checklist_generator.py
│   ├── qa_generator.py
│   └── booklet_generator.py
├── app/
│   ├── review_app.py
│   ├── preview_app.py
│   └── narrative_engine.py
├── samples/
│   ├── sample_training_asset_spec.json
│   ├── sample_manual.html
│   ├── sample_summary.txt
│   └── sample_checklist.json
├── archive/
│   ├── raw/
│   ├── evidence/
│   ├── structured/
│   └── deliverables/
```

```
├── tests/
│   ├── test_schema_validation.py
│   ├── test_structure_extractor.py
│   ├── test_pdf_generation.py
│   └── test_repository.py
├── requirements.txt
├── requirements-dev.txt
└── .env.example
```

14. 開発原則

本プロジェクトでは、以下の原則を重視します。

Single Source of Truth

構造化JSONを正本とし、PDFや要約は派生物として扱います。
レポート修正ではなく、元JSONの修正を優先します。

Evidence First

自動抽出結果には必ず証跡を持たせます。
時刻、OCR、画面情報、信頼度を保持し、再確認可能な設計にします。

Human-in-the-loop

完全自動化を前提にせず、人手レビューを品質保証の一部として組み込みます。
特に手順抜け、危険作業、例外処理は人手確認を前提とします。

Reusable Delivery

一度構造化した資産は、PDFだけでなくFAQ、チェックリスト、ダイジェスト、検索基盤などへ再利用可能であるべきです。

Format Separation

データ構造と表示形式を分離します。
JSON、HTML、PDF、Text の責務を明確に切り分け、差し替え可能にします。

15. 想定ユースケースとターゲット

想定ユースケース

- 新人研修
 - 動画だけでなく、手順書・要点資料として再利用
- SOP整備
 - 業務手順の標準化
 - 現場ごとの差分縮小
- FAQ前処理
 - 動画に含まれる質問・注意点・例外対応の素材化

- 動画棚卸し
 - 既存動画資産の再評価
 - 使える動画、統合すべき動画、陳腐化した動画の整理

想定ターゲット

- 研修動画・マニュアル動画を多く保有する企業
- 人事・教育担当
- 現場トレーナー
- 業務マニュアル担当
- OJT依存が強く、文書整備が進んでいない部署
- 動画はあるが、検索性・参照性・再利用性に課題を感じている組織

16. 次に決めるべきこと

初期立ち上げにあたり、次の項目を優先的に確定する必要があります。

- training_asset_spec の JSONスキーマ初版
 - 抽出プロンプト設計
 - 説明パートと手順パートの分類ルール
 - 注意点・NG例・チェックリスト抽出ルール
 - PDFテンプレート初版
 - 表紙
 - 目次
 - 手順セクション
 - 注意点セクション
 - チェックリストセクション
 - レビュー運用
 - 誰が確認するか
 - どの粒度で修正するか
 - 承認フローをどう持つか
 - 命名規則
 - asset_id
 - chapter_id
 - procedure_id
 - step_id
 - deliverableファイル名
 - 証跡保持ルール
 - どこまで時刻を保持するか
 - OCR保持単位
 - 画面キャプチャ候補の抽出基準
 - MVP対象動画の選定基準
 - 操作型
 - 講義型
 - 混合型
 - サンプル納品フォーマット
 - JSONサンプル
 - PDFサンプル
 - 要約サンプル
-

まとめ

VideoAsset Manualize は、既存の研修動画・マニュアル動画を、**教育資産・業務資産として再利用可能な形に変換するプロジェクト**です。

単なる文字起こしでも、単なる動画制作でもなく、**動画を現場で使える手順書・知識資産へ変換することを目指します。**

中核となるのは、動画そのものではなく、そこから生成される **構造化JSON** です。
この正本データを中心に、RAWログ、要約、HTML、PDF、チェックリスト、FAQ候補などへ展開することで、動画資産の価値を最大化します。

今後は、JSONスキーマ・抽出ルール・テンプレート・レビュー運用を順に固め、MVPから商品化へ進めていきます。

17. リポジトリ初期ドキュメントパック

本章では、**VideoAsset Manualize** を最短で「リポジトリとして動かせる状態」にするための**ドキュメント構成**を定義します。

現時点では本READMEを母艦ドキュメントとし、以降は責務ごとに docs/ 配下へ分割していく方針を推奨します。

17.1 このドキュメントパックの狙い

- READMEだけで全体像が分かる状態を作る
- 仕様、設計、MVP、テンプレートを役割ごとに切り出せるようにする
- 開発者、提案担当、レビュー担当が同じ前提で動けるようにする
- 「まだ何も決まっていない」状態から、着手順と成果物を明確にする

17.2 まず作るべきドキュメント一覧

パス	目的	優先度	状態の目安
README.md	全体概要、入口、目次	最優先	本文書を初版として採用
docs/concept/PROJECT_CHARTER.md	プロジェクト憲章、目的、範囲、非範囲	最優先	READMEから抜粋・整理
docs/concept/VALUE_PROPOSITION.md	提供価値、ターゲット、導入意義	高	提案資料と整合
docs/architecture/SYSTEM_OVERVIEW.md	アーキテクチャ全体像	最優先	パイプライン図と責務を定義
docs/architecture/EVIDENCE_MODEL.md	証跡モデルと保持方針	高	時刻・OCR・画面情報の定義
docs/architecture/PIPELINE_DESIGN.md	処理フロー詳細	高	入出力と処理段階を明記
docs/roadmap/MVP_PLAN.md	MVPの対象範囲、評価基準、進め方	最優先	パイロット実行用

パス	目的	優先度	状態の目安
		先	
docs/roadmap/PHASES.md	Phaseごとの計画	中	開発・提案の共通認識用
docs/templates/MANUAL_TEMPLATE_SPEC.md	マニュアルの章立て仕様	高	PDF/HTML両対応の元資料
docs/templates/PDF_LAYOUT_GUIDE.md	PDFレイアウト指針	中	デザイン強化前の基準
		最優先	
schemas/training_asset_spec.schema.json	正本JSONスキーマ	優先	実装着手の起点
		高	
samples/sample_training_asset_spec.json	サンプル出力	高	スキーマ確認・営業説明用

17.3 README と docs の役割分担

領域	README に書くこと	docs に分割すること
概要	目的、価値、立ち位置、全体像	詳細背景、提案観点、想定業界別の整理
設計	アーキテクチャ要約、データモデル要約	スキーマ詳細、パイプライン詳細、証跡仕様
実行計画	MVP概要、ロードマップ概要	Phase別タスク、受入基準、レビュー手順
出力物	納品物一覧、テンプレート方針	PDF設計ガイド、章立て詳細、サンプル
運用	開発原則、次に決めるべきこと	命名規則、承認フロー、更新ルール

17.4 本READMEを分割する際の推奨マッピング

本READMEの章	分割先の候補
1～5	docs/concept/PROJECT_CHARTER.md
6～9	docs/architecture/SYSTEM_OVERVIEW.md / PIPELINE_DESIGN.md
8 の証跡・レイヤー方針	docs/architecture/EVIDENCE_MODEL.md
10～12	docs/roadmap/MVP_PLAN.md / PHASES.md
13	docs/architecture/SYSTEM_OVERVIEW.md もしくは docs/PROJECT_STRUCTURE.md
14～16	docs/operations/WORKING_PRINCIPLES.md など
本章 17～20	初期整備ガイドとして README 残置 or docs/SETUP_DOC_PACK.md

18. リポジトリ整備の即実行プラン

素早く土台を固めるため、以下の順序で整備することを推奨します。

Step 1: README を入口として固定

最初にこのREADMEをリポジトリの正式な入口文書として確定します。

含めるべき最低要素:

- 何をするプロジェクトか
- なぜ必要か
- 何を作るか
- どのように進めるか
- 次に何を決めるか

Step 2: スキーマとサンプルを最優先で作成

実装と営業説明の両方に効くため、以下を最優先で作成します。

- `schemas/training_asset_spec.schema.json`
- `samples/sample_training_asset_spec.json`
- `docs/templates/MANUAL_TEMPLATE_SPEC.md`

これにより、出力物の完成イメージが曖昧なまま実装が進むことを防げます。

Step 3: パイプライン設計を文書化

動画取得からPDF生成までの責務分担を確定します。

最低限定義する内容:

- 入力形式
- 中間生成物
- 証跡モデル
- 手順抽出口ジックの責務
- HTML/PDF生成の責務
- 人手レビューの挿入ポイント

Step 4: MVPドキュメントを確定

PoCやパイロットで迷わないように、MVPの範囲と合格条件を固定します。

最低限必要な記載:

- 対象動画タイプ
- 1本あたりの成果物
- レビュー担当
- 合格基準
- フィードバック反映の方法

19. 初期Issue・マイルストーン案

GitHub運用を始める前提で、初期のIssue群とマイルストーンを先に定義しておく、着手が早くなります。

19.1 推奨マイルストーン

マイルストーン	目的	完了条件
M0: Project Foundation	README・構成・方針の固 定	README と docs の骨格が揃う

マイルストーン	目的	完了条件
M1: Schema & Template	スキーマとテンプレート初版の確定	JSONスキーマとマニュアルテンプレートがレビュー済み
M2: MVP Pipeline	1動画分の処理成立	JSON / RAW / 要約 / PDF が生成できる
M3: Pilot Review	現場レビュー反映	手順抜け・粒度・注意点の改善が完了
M4: Productization	提供パッケージ化	納品形式、価格帯、運用ガイドが整う

19.2 初期Issue候補

Foundation

- README初版の確定
- docs/ ディレクトリ構成の作成
- 命名規則案の策定
- .env.example の定義

Schema / Data

- training_asset_spec スキーマ初版
- transcript segment スキーマ定義
- procedure step スキーマ定義
- checklist item スキーマ定義
- サンプルJSON作成

Pipeline

- 動画ローダー方針の定義
- transcript pipeline のインターフェース定義
- OCR pipeline のインターフェース定義
- evidence builder の責務設計
- structure extractor の抽出ルール定義

Delivery

- HTMLテンプレート初版
- PDFレイアウト初版
- 要約テキスト出力フォーマット定義
- チェックリスト出力フォーマット定義

Review / Operations

- 人手レビュー項目の定義
- 現場確認シートの作成
- MVP受入基準の文書化
- 将来の冊子化要件の棚卸し

20. リポジトリ公開直後に揃えたい最低セット

最短で「見せられるリポジトリ」にするには、以下のセットが揃っている状態を目標にします。

20.1 Minimum Repository-Ready Set

- README.md に全体像とロードマップがある
- docs/ 配下に少なくとも 3 文書がある
 - PROJECT_CHARTER.md
 - SYSTEM_OVERVIEW.md
 - MVP_PLAN.md
- schemas/ に training_asset_spec.schema.json がある
- samples/ にサンプルJSONがある
- requirements.txt または今後の予定が明記されている
- .env.example の方針がある

20.2 初回コミットの推奨構成

```
Initial repository-ready commit
├── README.md
├── docs/concept/PROJECT_CHARTER.md
├── docs/architecture/SYSTEM_OVERVIEW.md
├── docs/roadmap/MVP_PLAN.md
├── schemas/training_asset_spec.schema.json
├── samples/sample_training_asset_spec.json
└── .env.example
```

20.3 Done の判断基準

以下を満たせば、初期のドキュメントパックとして十分に成立しているとみなせます。

- 第三者が README を読んでプロジェクトの目的を説明できる
- 開発者が最初に何を作るべきか判断できる
- 提案担当が価値と納品物を説明できる
- レビュー担当が何を確認すべきか把握できる
- 今後のスキーマ作成と MVP 実装に迷いが減っている

20.4 次の具体アクション

ドキュメントパック整備後、次は以下の順で着手するのが効率的です。

1. training_asset_spec.schema.json の初版作成
2. sample_training_asset_spec.json の作成
3. MANUAL_TEMPLATE_SPEC.md の作成
4. MVP対象動画の選定
5. 抽出プロンプト仕様の作成
6. HTML → PDF テンプレートの初版作成

付録A. 初期ドキュメント作成順序の推奨

順位	ドキュメント	理由
1	README.md	リポジトリの入口になるため
2	training_asset_spec.schema.json	実装と出力仕様の基準になるため
3	sample_training_asset_spec.json	完成イメージを共有しやすいため
4	MVP_PLAN.md	スコープ逸脱を防ぐため
5	SYSTEM_OVERVIEW.md	責務分担を明確化するため
6	MANUAL_TEMPLATE_SPEC.md	PDF/HTMLの実装基準になるため

付録B. このREADMEの使い方

- 現段階では **総合設計メモ兼README** として扱う
- 章単位で docs/ に分割し、README は要約版へ縮約していく
- スキーマとサンプルが固まったら、README は「概要 + 導線」に寄せる
- 詳細仕様は docs/architecture/、進行計画は docs/roadmap/ に逃がす

これにより、初期はスピード重視で一枚岩の文書として運用しつつ、実装が進んだ段階で自然に保守しやすい構成へ移行できます。